

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



**Department of
Electrical & Computer Engineering**

Lab Reports

Course Code: ECE 4212

Course Title: Computer Network Sessional

Submitted to:

Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:

Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Even

Submission Date: April 19, 2026

Table of Contents

Serial No.	Name of the Experiment
1	Study of Different Kinds of Topologies and Their Simulations
2	Study of Router-to-Router Connection and Static Routing Simulations
3	Implementation of Dynamic Routing using Routing Information Protocol (RIP)
4	Implementation of Static and RIP Routing in a Loop Configuration & Simulations
5	Study of Static Routing Configuration Using VLSM Technique and Simulation
6	Design and Implementation of Error Detection and Correction Mechanisms using CRC and Hamming Codes

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



**Department of
Electrical & Computer Engineering**

Lab Report 1

Study of Different Kinds of Topologies and Their Simulations

Course Code: ECE 4212

Course Title: Computer Network Sessional

Submitted to:

Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:

Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Odd

Submission Date: January 18, 2026

Contents

1	Introduction	1
1.1	Cisco Packet Tracer	1
2	Methodology	1
2.1	Point-to-Point (P2P) Topology	1
2.2	Bus Topology	2
2.3	Ring Topology	3
2.4	Star Topology (with Switch)	3
2.5	Star Topology (with Router)	4
2.6	Tree Topology	5
2.7	Mesh Topology	5
2.8	Hybrid Topology (Star-Ring)	6
3	Discussion	7
4	Conclusion	7
	References	7

Study of Different Kinds of Topologies and Their Simulations

1 Introduction

Network topology represents the geometric arrangement of various elements, such as links and nodes, within a computer network [1]. Selecting an appropriate topology is a fundamental step in network design, as it directly dictates the network's overall performance, cost-efficiency, scalability, and fault tolerance. In this sessional lab, we explore a comprehensive range of configurations—specifically Point-to-Point, Bus, Ring, Star, Tree, Mesh, and Hybrid topologies—to understand their operational strengths and limitations in a controlled environment.

1.1 Cisco Packet Tracer

Cisco Packet Tracer [2] is a sophisticated network simulation program developed by Cisco Systems to facilitate the experimentation of complex network behaviors. It provides a rich visual environment where users can design, configure, and troubleshoot virtual hardware, including routers, switches, and workstations. For the purpose of this report, Packet Tracer served as the primary tool to simulate real-time data flow via ICMP packets, allowing for the verification of end-to-end connectivity and path analysis through its "Simulation Mode."

2 Methodology

The methodology of this lab involves the systematic construction and testing of seven distinct network architectures. Each topology is evaluated through a three-step validation process: first, the creation of a physical **Schematic**; second, the execution of **Message Passing** to confirm successful delivery; and third, the analysis of the **Event List**, which captures the precise hop-by-hop journey of data through the network infrastructure.

2.1 Point-to-Point (P2P) Topology

This topology consists of a direct link between two endpoints, ideal for secure communication like leased lines. It offers simplicity, low cost, and guaranteed bandwidth but lacks scalability and redundancy.

The Cisco Packet Tracer simulation shows two workstations connected via Ethernet, with the schematic confirming successful ICMP pings, validating connectivity.

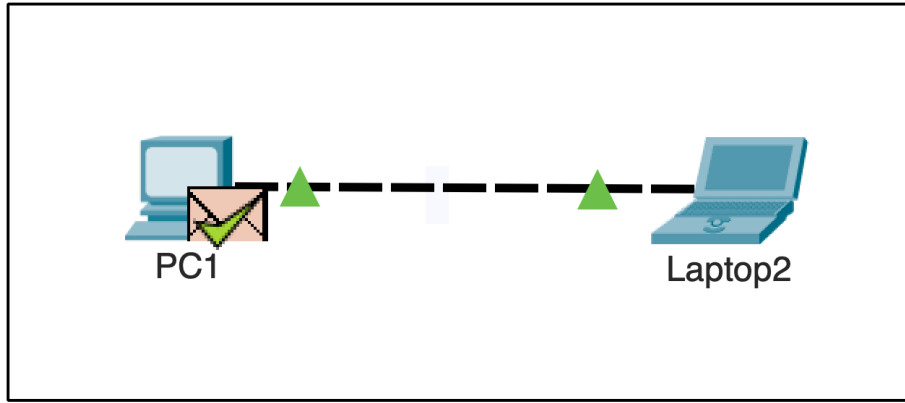


Figure 1: P2P Schema

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC1	Laptop2	ICMP		0.000	N	0	(edit)	(delete)

Figure 2: P2P Successful Message Passing

2.2 Bus Topology

A Bus topology is one of the simplest network configurations where all nodes—such as computers, printers, and servers—are connected to a single central cable, often called the "backbone." This backbone serves as the shared communication medium; data transmitted by any node travels along the cable and is received by all other nodes, though only the intended recipient processes the packet. While cost-effective and easy to set up for small networks, it is susceptible to network-wide failure if the central cable is damaged [3].

The following figures illustrate the network simulation: Figure 3 shows the physical arrangement and the sequence of events during data transmission, while Figure 5 confirms the successful delivery of the simulated message.

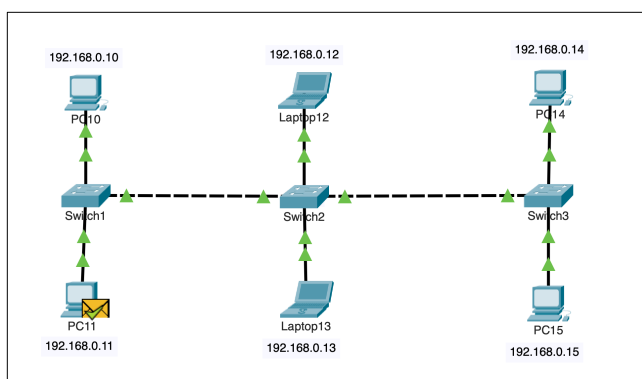


Figure 3: Bus Topology Schematic

Simulation Panel		
Event List		
Vis.	Time(sec)	Last Device
	0.000	--
	0.001	PC11
	0.002	Switch1
	0.003	Switch2
	0.004	Switch3
	0.005	PC14
	0.006	Switch3
	0.007	Switch2
Visible	0.008	Switch1

Figure 4: Bus Event List

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC11	PC14	ICMP		0.000	N	0	(edit)	(delete)

Figure 5: Bus Topology: Successful ICMP Message Passing

2.3 Ring Topology

In a Ring topology, each node is connected to exactly two other nodes, forming a continuous circular pathway for signals. Data travels from node to node in one direction (unidirectional) or sometimes two (bidirectional), with each intermediate node acting as a repeater to keep the signal strong. This topology handles high-volume traffic better than a bus network and eliminates the need for a central server to manage connectivity between workstations [3].

The simulation of this topology is documented below. Figure 6 provides a side-by-side view of the circular node arrangement and the chronological event list of the packet's journey. Figure 8 displays the ICMP status table, verifying that the packet successfully traversed the ring to its destination.

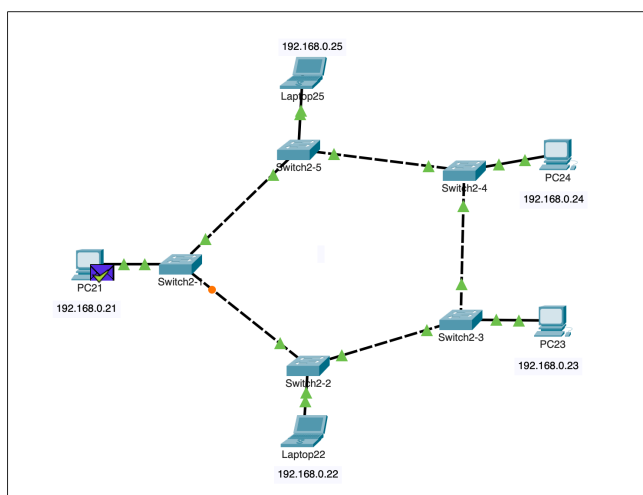


Figure 6: Ring Topology Schematic

Event List		
Vis.	Time(sec)	Last Device
	0.000	--
	0.012	--
	0.013	PC21
	0.014	Switch2-1
	0.015	Switch2-5
	0.016	Switch2-4
	0.017	Switch2-3
	0.018	Switch2-2
	0.019	Laptop22
	0.020	Switch2-2
	0.021	Switch2-3
	0.022	Switch2-4
	0.023	Switch2-5
Visible	0.024	Switch2-1

Figure 7: Ring Event List

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC21	Laptop22	ICMP		0.000	N	0	(edit)	(delete)

Figure 8: Ring Topology: Successful ICMP Message Passing

2.4 Star Topology (with Switch)

The Star topology is the most widely used configuration in modern Ethernet networks. In this setup, every device is connected to a central switch via a dedicated cable. The switch manages the data flow by directing packets only to the target device, which minimizes collisions. If one cable fails, only the node connected to that cable is affected, making the network highly resilient and easy to troubleshoot [3].

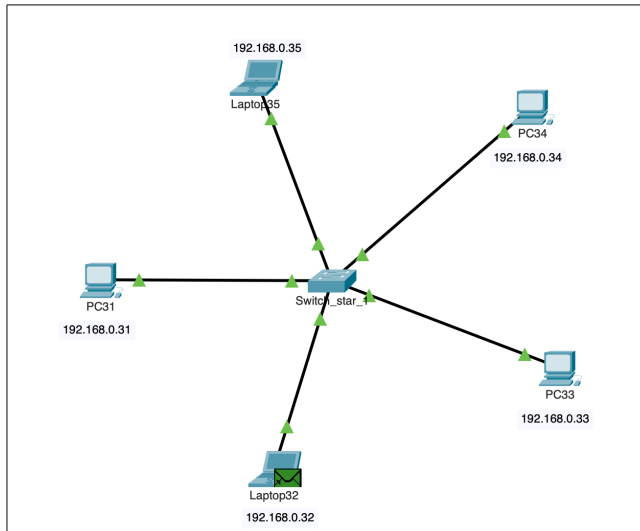


Figure 9: Star (Switch) Schematic

Event List		
Vis.	Time(sec)	Last Device
	0.000	--
	0.001	Laptop32
	0.002	Switch_star_1
	0.003	PC34
Visible	0.004	Switch_star_1

Figure 10: Star (Switch) Event List

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	Lapt...	PC34	ICMP		0.000	N	0	(edit)	(delete)

Figure 11: Star (Switch): Successful Message Passing

2.5 Star Topology (with Router)

Using a Router as the central hub of a Star topology adds a layer of intelligence to the network, enabling routing between different IP subnets. While a switch connects devices within the same network, the router handles communication between distinct network segments. This setup is typical for small office or home office (SOHO) environments where a single router provides connectivity for multiple local devices to the internet or other networks [3].

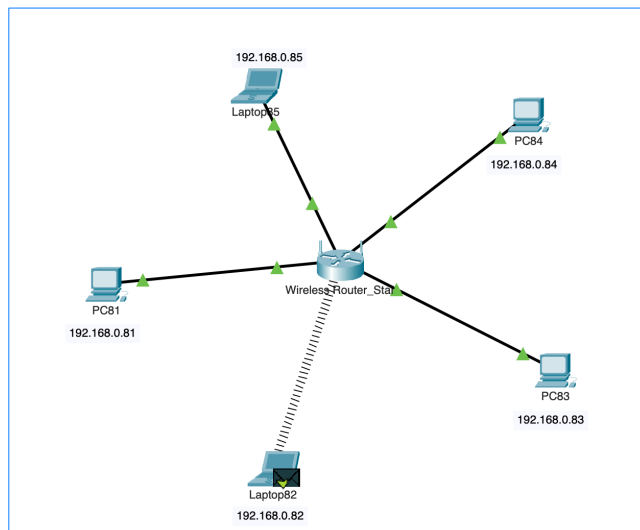


Figure 12: Star (Router) Schematic

Event List		
Vis.	Time(sec)	Last Device
	0.000	--
	0.012	--
	0.015	--
	0.016	Laptop82
	0.017	Wireless Router_Star
	0.017	--
	0.018	Wireless Router_Star
	0.018	Laptop85
Visible	0.019	Wireless Router_Star

Figure 13: Star (Router) Event List

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	Laptop82	Laptop85	ICMP		0.000	N	0	(edit)	(delete)

Figure 14: Star (Router): Successful Message Passing

2.6 Tree Topology

The Tree topology is a hierarchical structure where central nodes of various star networks are connected to a primary bus or a "root" switch. This creates a tiered architecture, typically consisting of a core layer, distribution layer, and access layer. It is highly scalable, allowing for the easy addition of new branches without disrupting the entire network. However, if the root node or the main backbone fails, the entire branch connected to it loses connectivity [3].

The simulation results are presented below. Figure 15 displays the hierarchical arrangement of switches and the resulting event log as the packet moves through the branches. Figure 17 provides the confirmation of successful ICMP communication between the chosen source and destination nodes.

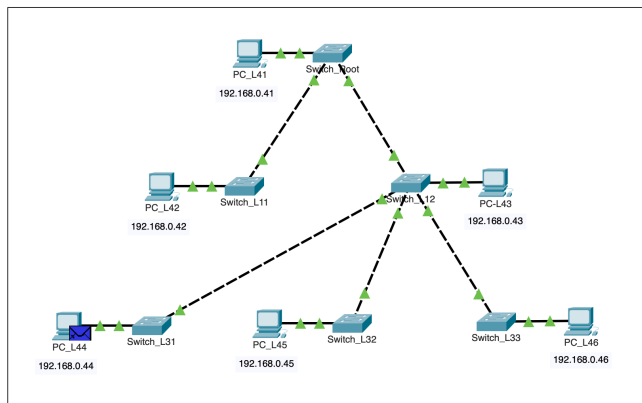


Figure 15: Tree Topology Schematic

Event List		
Vis.	Time(sec)	Last Device
	0.000	--
	0.001	PC_L44
	0.002	Switch_L31
	0.003	Switch_L12
	0.004	Switch_Root
	0.005	PC_L41
	0.006	Switch_Root
	0.007	Switch_L12
Visible	0.008	Switch_L31

Figure 16: Tree Event List

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC_L44	PC_L41	ICMP		0.000	N	0	(edit)	(delete)

Figure 17: Tree Topology: Successful Message Passing

2.7 Mesh Topology

A Mesh topology is a robust network design where nodes are interconnected, providing multiple redundant paths for data transmission. In a full mesh, every node is connected to every other node, while a partial mesh (as simulated here) connects key nodes to ensure that even if a specific link or switch fails, the network can reroute traffic through an alternative path. This topology offers the highest level of fault tolerance and is commonly used in critical backbone infrastructures and wireless mesh networks [3].

The simulation results, based on the interconnected switch fabric, are shown below. Figure 18 demonstrates the complex schematic and the detailed event list showing how packets are handled across multiple switch hops. Figure 20 displays the ICMP success table for the transfers between PC51 to PC53 and Laptop55 to Laptop52.

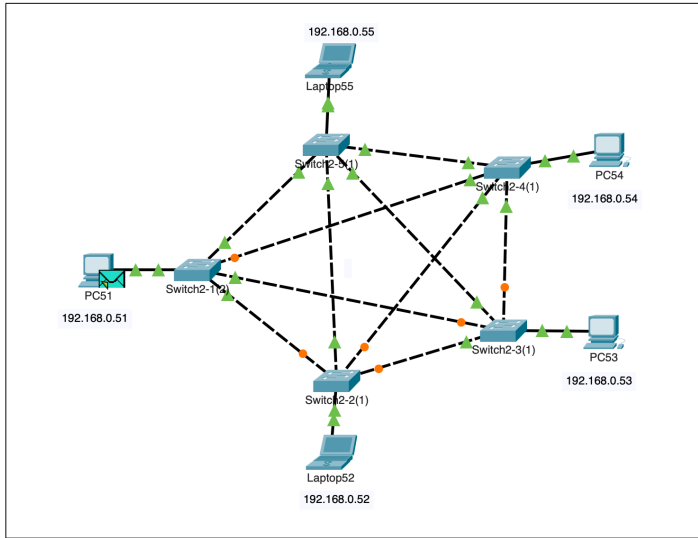


Figure 18: Mesh Topology Schematic

Event List		
Vis.	Time(sec)	Last Device
	0.000	--
	0.000	--
	0.006	--
	0.007	Laptop55
	0.008	Switch2-5(1)
	0.008	--
	0.009	PC51
	0.009	Switch2-2(1)
	0.010	Switch2-1(2)
	0.010	Laptop52
	0.011	Switch2-5(1)
	0.011	Switch2-2(1)
	0.012	Switch2-3(1)
	0.012	Switch2-5(1)
	0.013	PC53
	0.014	Switch2-3(1)
	0.015	Switch2-5(1)
Visible	0.016	Switch2-1(2)

Figure 19: Mesh Event List

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC51	PC53	ICMP		0.000	N	0	(edit)	(delete)
	Successful	Laptop55	Laptop52	ICMP		0.000	N	1	(edit)	(delete)

Figure 20: Mesh Topology: Successful Message Passing

2.8 Hybrid Topology (Star-Ring)

A Hybrid topology is an integration of two or more different topologies. In this specific simulation, a Star topology and a Ring topology are interconnected. This approach allows a network to leverage the centralized management of a Star network for one department while using the efficient, high-traffic handling of a Ring network for another. Hybrid networks are highly flexible and are typically found in large corporate environments where different floors or buildings use different local configurations but must remain part of a single cohesive network [3].

The following figures illustrate the combination of these two architectures. Figure 21 shows the schematic where a central switch (Star) links into a circular node arrangement (Ring), alongside the simulation's event list. Figure 23 verifies that packets successfully traverse between these two distinct structural segments.

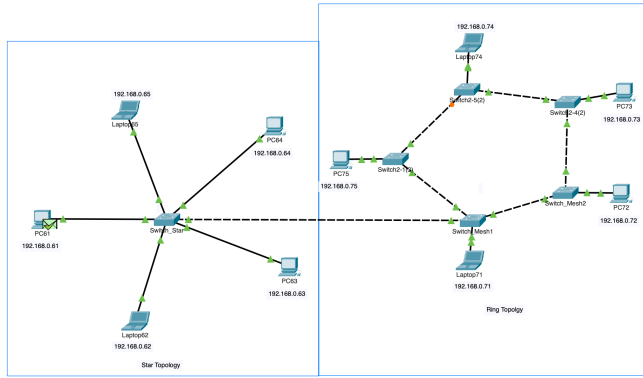


Figure 21: Hybrid (Star-Ring) Schematic

Event List		
Vis.	Time(sec)	Last Device
	0.000	--
	0.001	PC61
	0.002	Switch_Star
	0.003	Switch_Mesh1
	0.004	Switch_Mesh2
	0.005	Switch2-4(2)
	0.006	PC73
	0.007	Switch2-4(2)
	0.008	Switch_Mesh2
	0.009	Switch_Mesh1
Visible	0.010	Switch_Star

Figure 22: Hybrid Event List

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC61	PC73	ICMP		0.000	N	0	(edit)	(delete)

Figure 23: Hybrid Topology: Successful Message Passing

3 Discussion

The simulations conducted in this lab highlight the trade-offs inherent in network design. While simpler topologies like Bus and Star are cost-effective and straightforward to implement, they are vulnerable to single points of failure—either the backbone cable or the central hub. In contrast, the Mesh and Tree topologies demonstrated superior scalability and reliability, with the Mesh setup offering the highest fault tolerance due to its redundant paths.

4 Conclusion

Through this sessional lab, we successfully simulated and analyzed seven diverse network topologies. By observing packet propagation across different mediums and examining how central devices like switches and routers manage traffic, we gained a practical understanding of network architecture. The use of Cisco Packet Tracer provided a realistic and iterative environment to bridge the gap between theoretical networking concepts and practical implementation.

References

- [1] W. Stallings, “Local and metropolitan area networks,” *Prentice Hall*, 2013, detailed analysis of Bus, Ring, and Star configurations.
- [2] *Cisco Packet Tracer: Simulation-Based Learning Tool*, Cisco Networking Academy, 2024. [Online]. Available: <https://www.netacad.com/courses/packet-tracer>
- [3] A. S. Tanenbaum, N. Feamster, and D. J. Wetherall, *Computer Networks*, 6th ed. Pearson, 2021, general overview of network topologies and the OSI model.

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



**Department of
Electrical & Computer Engineering**

Lab Report 2

Study of Router-to-Router Connection and Static Routing Simulations

Course Code: ECE 4212

Course Title: Computer Network Sessional

Submitted to:

Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:

Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Odd

Submission Date: February 1, 2026

Contents

1	Theory and Introduction	1
2	Methodology	1
3	Results	2
4	Discussion	3
5	Conclusion	3
	References	3

Study of Router-to-Router Connection and Static Routing Simulations

1 Theory and Introduction

Routing is the fundamental process of directing data packets across an interconnected network from a source node to a destination node through various intermediate devices, primarily routers. This experiment focuses on the Network Layer (Layer 3) of the Open Systems Interconnection (OSI) model, where logical addressing and path determination occur [1]. Routers serve as the gateways between different subnets; however, they only possess knowledge of networks to which they are physically and directly connected by default.

To facilitate communication across non-adjacent subnets, static routing was implemented. Static routing involves the manual configuration of fixed paths in the routing table by a network administrator [2]. Unlike dynamic protocols, static routes do not change unless manually reconfigured, making them highly predictable and suitable for smaller topologies where bandwidth overhead must be minimized. The primary mechanism used in this setup is the "Next Hop" logic, which identifies the IP address of the adjacent router's interface as the entry point for data destined for a remote network. This ensures that even when multiple paths exist, data follows a deterministic route defined by the administrator.

2 Methodology

The experiment was conducted in a virtualized environment using Cisco Packet Tracer (Version 9.0) [3]. The following expanded steps were executed to establish and validate the network:

- **Topology Construction:** A multi-router architecture was designed consisting of three routers (Router0, Router1, and Router2), three switches, and six end-devices, including PCs and laptops.
- **Addressing Scheme:** Three distinct Class A IP subnets (10.0.0.0, 20.0.0.0, and 30.0.0.0) were assigned to the local area networks (LANs) via FastEthernet interfaces.
- **WAN Configuration:** High-speed Serial interfaces (Se2/0 and Se3/0) were utilized to connect the routers, forming two Wide Area Network (WAN) segments using the 40.0.0.0 and 50.0.0.0 subnets. Clock rates were set on the DCE ends of the serial cables to synchronize data transmission.
- **Static Route Implementation:** Manual routing entries were programmed into each router's global configuration mode using the `ip route` command. For instance, Router 0 was configured with routes to the 20.0.0.0 and 30.0.0.0 networks via the next-hop address 40.40.40.2.
- **Path Validation:** Connectivity was systematically verified using the Internet Control Message Protocol (ICMP) *Ping* utility. To analyze the precise hop-by-hop traversal and confirm that data followed the manually defined paths, the *Tracert* command was executed from end-devices.

3 Results

The addressing scheme and device configurations are summarized in Table 1.

Table 1: Network Addressing Table

Device	Interface	IPv4 Address	Subnet Mask	Default Gateway
Router 0	Fa0/0	10.10.10.0	255.0.0.0	N/A
Router 0	Se2/0	40.40.40.1	255.0.0.0	N/A
Router 1	Fa0/0	20.20.20.0	255.0.0.0	N/A
Router 1	Se2/0	40.40.40.2	255.0.0.0	N/A
Router 1	Se3/0	50.50.50.1	255.0.0.0	N/A
Router 2	Fa0/0	30.30.30.0	255.0.0.0	N/A
Router 2	Se2/0	50.50.50.2	255.0.0.0	N/A
PC 11	NIC	10.10.10.1	255.0.0.0	10.10.10.0
PC 21	NIC	20.20.20.1	255.0.0.0	20.20.20.0
PC 32	NIC	30.30.30.2	255.0.0.0	30.30.30.0

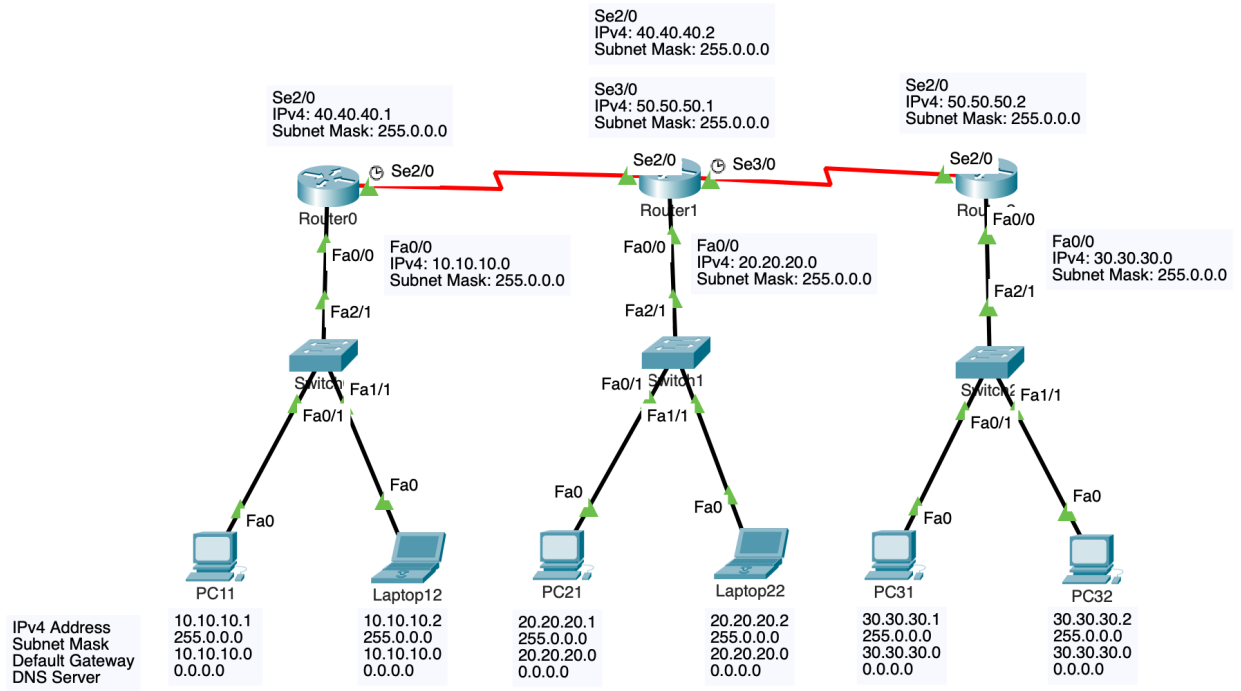


Figure 1: Network Topology with Multi-Router Configuration

```

Router>enable
Router#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       I - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

C    10.0.0.0/8 is directly connected, FastEthernet0/0
S    20.0.0.0/8 [1/0] via 40.40.40.2
S    30.0.0.0/8 [1/0] via 40.40.40.2
C    40.0.0.0/8 is directly connected, Serial2/0

Router#

```

Figure 2: Router 0 CLI: Static and Connected Routes

```

Cisco Packet Tracer PC Command Line 1.0
C:\>tracert 30.30.30.2

Tracing route to 30.30.30.2 over a maximum of 30 hops:

  0  3 ms    0 ms    0 ms    10.10.10.0
  1  39 ms    2 ms    44 ms   40.40.40.2
  2  57 ms    60 ms   20 ms   50.50.50.2
  3  28 ms    1 ms    1 ms    30.30.30.2

Trace complete.

C:\>

```

Figure 3: Tracert Path Analysis from 10.10.10.1 to 30.30.30.2

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC11	Laptop12	ICMP		0.000	N	0	(...)	(delete)
	Failed	PC11	PC21	ICMP		0.000	N	1	(...)	(delete)
	Successful	PC11	PC21	ICMP		0.000	N	2	(...)	(delete)
	Failed	Laptop12	PC32	ICMP		0.000	N	3	(...)	(delete)
	Successful	Laptop12	PC32	ICMP		0.000	N	4	(...)	(delete)

Figure 4: Verification of Connectivity and Address Resolution Latency

4 Discussion

The successful implementation of the network was confirmed through detailed analysis of the routing tables and path tracing. As observed in Figure 2, the routing table for Router 0 contains both directly connected routes (labeled 'C') and manually configured static routes (labeled 'S'). The entry `S 30.0.0.0/8 [1/0] via 40.40.40.2` indicates that traffic destined for the 30.0.0.0 network must be forwarded to Router 1 at 40.40.40.2. A critical observation was the "Failed" status of the first ICMP packet (Figure 4), resulting from Address Resolution Protocol (ARP) timeout. When PC11 pings a device in another subnet, it must resolve the MAC address of its default gateway. The ICMP request is held while the ARP cycle completes, causing the initial ping to expire. Subsequent pings succeed because the gateway's MAC address is cached. The path analysis using *tracert* (Figure 3) verified the routing logic, showing four hops from the local gateway through intermediate routers to the destination, confirming that static routes were correctly executed.

5 Conclusion

The study of router-to-router connectivity through static routing was successfully completed. The implementation demonstrated that while static routing is manageable for small scales, it requires precise "Next Hop" configurations to maintain path integrity. The use of Cisco Packet Tracer provided a realistic environment to observe background control-plane protocols like ARP and their impact on data-plane traffic.

References

- [1] A. S. Tanenbaum, N. Feamster, and D. J. Wetherall, *Computer Networks*, 6th ed. Pearson, 2021.
- [2] W. Stallings, *Data and Computer Communications*, 10th ed. Pearson, 2013.
- [3] *Cisco Packet Tracer 9.0 User Guide*, Cisco Networking Academy, 2026.

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



**Department of
Electrical & Computer Engineering**

Lab Report 3

Implementation of Dynamic Routing using Routing Information Protocol (RIP)

Course Code: ECE 4212

Course Title: Computer Network Sessional

Submitted to:

Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:

Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Odd

Submission Date: February 1, 2026

Contents

1	Theory and Introduction	1
2	Methodology	1
3	Results	2
4	Discussion	3
5	Conclusion	3
	References	3

Implementation of Dynamic Routing using Routing Information Protocol (RIP)

1 Theory and Introduction

Routing is the fundamental process of directing data packets across an interconnected network from a source node to a destination node through various intermediate devices. This experiment focuses on the Network Layer (Layer 3) of the Open Systems Interconnection (OSI) model [1]. While static routing is effective for small, unchanging topologies, it becomes administratively burdensome as networks scale.

Dynamic routing addresses this by allowing routers to automatically exchange information about the network topology. The Routing Information Protocol (RIP) is a distance-vector protocol that utilizes the Bellman-Ford algorithm to determine the best path based on hop count.

In this experiment, RIP was implemented. RIP supports Variable Length Subnet Masking (VLSM) and utilizes multicast addressing for routing updates, which prevents unnecessary processing by non-routing devices on the network [2].

2 Methodology

The experiment was conducted using Cisco Packet Tracer (Version 9.0) [3]. The topology from the previous experiment was extended and converted to a dynamic routing environment through the following steps:

- **Topology Extension:** A fourth router (Router 3) and a corresponding LAN were integrated into the existing network. New serial links were established to connect Router 2 to Router 3.
- **Removal of Static Routes:** All previously configured manual routes were deleted using the `no ip route` command to ensure that the dynamic protocol had full control over the routing table.
- **RIP Activation:** The dynamic routing process was initiated on all routers using the `router rip` command. The version was explicitly set to `version 2`.
- **Network Advertisement:** Each router was configured to "advertise" its directly connected networks using the `network` command. For example, Router 0 advertised 10.0.0.0 and 40.0.0.0.
- **Verification:** Connectivity was verified across all four subnets using the *Ping* and *Tracert* utilities to ensure the routers had successfully synchronized their tables.

Table 1: Extended Network Addressing Table (RIPv2)

Device	Interface	IPv4 Address	Subnet Mask	Default Gateway
Router 0	Fa0/0	10.10.10.0	255.0.0.0	N/A
Router 0	Se2/0	40.40.40.1	255.0.0.0	N/A
Router 1	Fa0/0	20.20.20.0	255.0.0.0	N/A
Router 1	Se2/0	40.40.40.2	255.0.0.0	N/A
Router 1	Se3/0	50.50.50.1	255.0.0.0	N/A
Router 2	Fa0/0	30.30.30.0	255.0.0.0	N/A
Router 2	Se2/0	50.50.50.2	255.0.0.0	N/A
Router 2	Se3/0	70.70.70.1	255.0.0.0	N/A
Router 3	Fa0/0	60.60.60.0	255.0.0.0	N/A
Router 3	Se2/0	70.70.70.2	255.0.0.0	N/A
PC 11	NIC	10.10.10.1	255.0.0.0	10.10.10.0
Laptop 12	NIC	10.10.10.2	255.0.0.0	10.10.10.0
PC 21	NIC	20.20.20.1	255.0.0.0	20.20.20.0
Laptop 22	NIC	20.20.20.2	255.0.0.0	20.20.20.0
PC 31	NIC	30.30.30.1	255.0.0.0	30.30.30.0
PC 32	NIC	30.30.30.2	255.0.0.0	30.30.30.0
PC 41	NIC	60.60.60.1	255.0.0.0	60.60.60.0
Laptop 42	NIC	60.60.60.2	255.0.0.0	60.60.60.0

3 Results

The comprehensive addressing scheme for the extended network is summarized in Table 1.

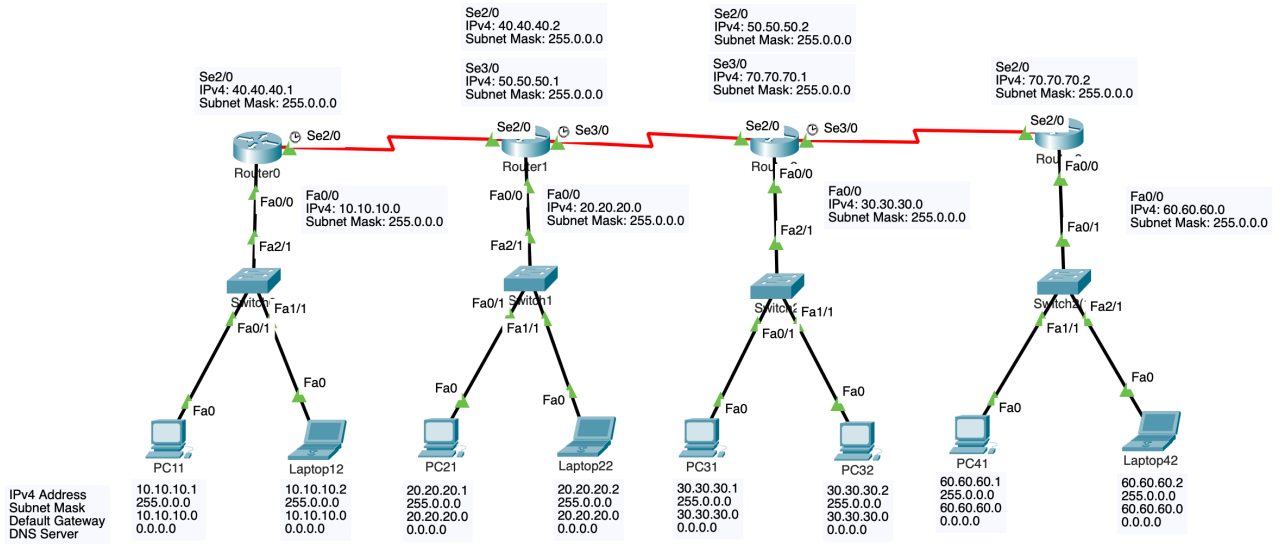


Figure 1: Extended Topology with 4 Routers and Dynamic RIP Routing

```
%SYS-5-CONFIG_I: Configured from console by console

Router#enable
Router#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

R    10.0.0.0/8 [120/2] via 50.50.50.1, 00:00:07, Serial2/0
R    20.0.0.0/8 [120/1] via 50.50.50.1, 00:00:07, Serial2/0
C    30.0.0.0/8 is directly connected, FastEthernet0/0
R    40.0.0.0/8 [120/1] via 50.50.50.1, 00:00:07, Serial2/0
C    50.0.0.0/8 is directly connected, Serial2/0
R    60.0.0.0/8 [120/1] via 70.70.70.2, 00:00:26, Serial3/0
C    70.0.0.0/8 is directly connected, Serial3/0
```

Figure 2: Routing Table showing RIP Learned Routes (Marked with 'R')

4 Discussion

The primary observation in this experiment was the efficiency of network convergence. Upon adding the 70.0.0.0 network, the information propagated automatically through the neighbors. As demonstrated in Figure 2, the routing table entries were updated to reflect the new topology without manual intervention.

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC11	Laptop12	IC...		0.000	N	0	(...)	(delete)
	Failed	PC11	Laptop22	IC...		0.000	N	1	(...)	(delete)
	Successful	PC11	Laptop22	IC...		0.000	N	2	(...)	(delete)
	Failed	Laptop42	PC31	IC...		0.000	N	3	(...)	(delete)
	Successful	Laptop42	PC31	IC...		0.000	N	4	(...)	(delete)

Figure 3: Verification of End-to-End Connectivity via Successive ICMP Requests

In the *tracert* analysis from PC11 to PC41, the packet traversed three intermediate routers. This confirmed that RIP correctly calculated the shortest path based on the minimum hop count. The "Initial Failure" phenomenon was again observed due to ARP resolution (Figure 3), but subsequent communication was seamless. This experiment highlights that while RIPv2 is limited by a maximum hop count of 15, its ease of configuration makes it superior to static routing for expanding networks.

5 Conclusion

The implementation of dynamic routing using RIPv2 was successfully achieved. The experiment demonstrated the protocol's ability to automate routing table updates and manage multi-hop communication without manual path definition. By comparing this to the previous static routing lab, the scalability and reduced administrative overhead of dynamic protocols were clearly observed.

References

- [1] A. S. Tanenbaum, N. Feamster, and D. J. Wetherall, *Computer Networks*, 6th ed. Pearson, 2021.
- [2] W. Stallings, *Data and Computer Communications*, 10th ed. Pearson, 2013.
- [3] *Cisco Packet Tracer 9.0 User Guide*, Cisco Networking Academy, 2026.

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



**Department of
Electrical & Computer Engineering**

Lab Report 4

**Implementation of Static and RIP Routing in a Loop Configuration &
Simulations**

Course Code: ECE 4212

Course Title: Computer Network Sessional

Submitted to:

Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:

Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Odd

Submission Date: April 5, 2026

Contents

1	Theory and Introduction	1
2	Methodology	1
3	Results	2
4	Discussion	5
5	Conclusion	6
	References	6

Implementation of Static and RIP Routing in a Loop Configuration & Simulations

1 Theory and Introduction

Routing is the fundamental process of directing data packets across an interconnected network from a source node to a destination node through various intermediate devices. This experiment focuses on the Network Layer (Layer 3) of the Open Systems Interconnection (OSI) model [1].

While linear or tree topologies are straightforward, they suffer from single points of failure. A loop topology introduces physical redundancy by providing multiple paths between any two nodes. In static routing, leveraging this redundancy requires the manual configuration of multiple pathways (e.g., equal-cost load balancing or floating static routes) so that if a primary link fails, an alternate path is predefined [2].

Dynamic routing addresses this automatically. The Routing Information Protocol (RIP) utilizes the Bellman-Ford algorithm to determine the best path based on hop count. In a loop configuration, if an active link fails, RIP dynamically recalculates the routing table and reroutes traffic through the surviving alternate paths without requiring manual administrative intervention, thereby providing superior fault tolerance [1].

2 Methodology

The experiment was conducted using Cisco Packet Tracer (Version 9.0) [3]. A redundant loop topology was designed and tested in both static and dynamic environments through the following steps:

- **Topology Construction:** A loop network was constructed utilizing three routers (Router0, Router1, Router2) interconnected via serial links (40.0.0.0, 50.0.0.0, and 70.0.0.0 networks). Three Local Area Networks (10.0.0.0, 20.0.0.0, and 30.0.0.0) were connected to the respective routers.
- **Phase 1 - Static Routing:** Static routes were manually configured. To utilize the loop's redundancy, multiple "Next Hop" routes were assigned for the same destination to facilitate alternative pathing.
- **Phase 2 - RIP Integration:** The static routes were removed, and the dynamic RIP routing process (`version 2`) was activated on all routers using the `network` command to advertise directly connected subnets.
- **Phase 3 - Fault Simulation:** A physical link failure was simulated by bringing down the connection between Router0 and Router1 in the RIP environment.
- **Verification:** End-to-end connectivity was evaluated using the *Ping* utility in the static state, the optimal RIP state, and the faulty RIP state to verify network convergence and resilience.

3 Results

The comprehensive addressing scheme for the loop network is summarized in Table 1.

Table 1: Loop Network Addressing Table

Device	Interface	IPv4 Address	Subnet Mask	Default Gateway
Router 0	Fa0/0	10.10.10.0	255.0.0.0	N/A
Router 0	Se2/0	40.40.40.1	255.0.0.0	N/A
Router 0	Se3/0	70.70.70.1	255.0.0.0	N/A
Router 1	Fa0/0	20.20.20.0	255.0.0.0	N/A
Router 1	Se2/0	40.40.40.2	255.0.0.0	N/A
Router 1	Se3/0	50.50.50.1	255.0.0.0	N/A
Router 2	Fa0/0	30.30.30.0	255.0.0.0	N/A
Router 2	Se2/0	50.50.50.2	255.0.0.0	N/A
Router 2	Se3/0	70.70.70.2	255.0.0.0	N/A
PC 11	NIC	10.10.10.1	255.0.0.0	10.10.10.0
Laptop 12	NIC	10.10.10.2	255.0.0.0	10.10.10.0
PC 13	NIC	10.10.10.3	255.0.0.0	10.10.10.0
PC 21	NIC	20.20.20.1	255.0.0.0	20.20.20.0
Laptop 22	NIC	20.20.20.2	255.0.0.0	20.20.20.0
PC 23	NIC	20.20.20.3	255.0.0.0	20.20.20.0
PC 31	NIC	30.30.30.1	255.0.0.0	30.30.30.0
PC 32	NIC	30.30.30.2	255.0.0.0	30.30.30.0
PC 33	NIC	30.30.30.3	255.0.0.0	30.30.30.0

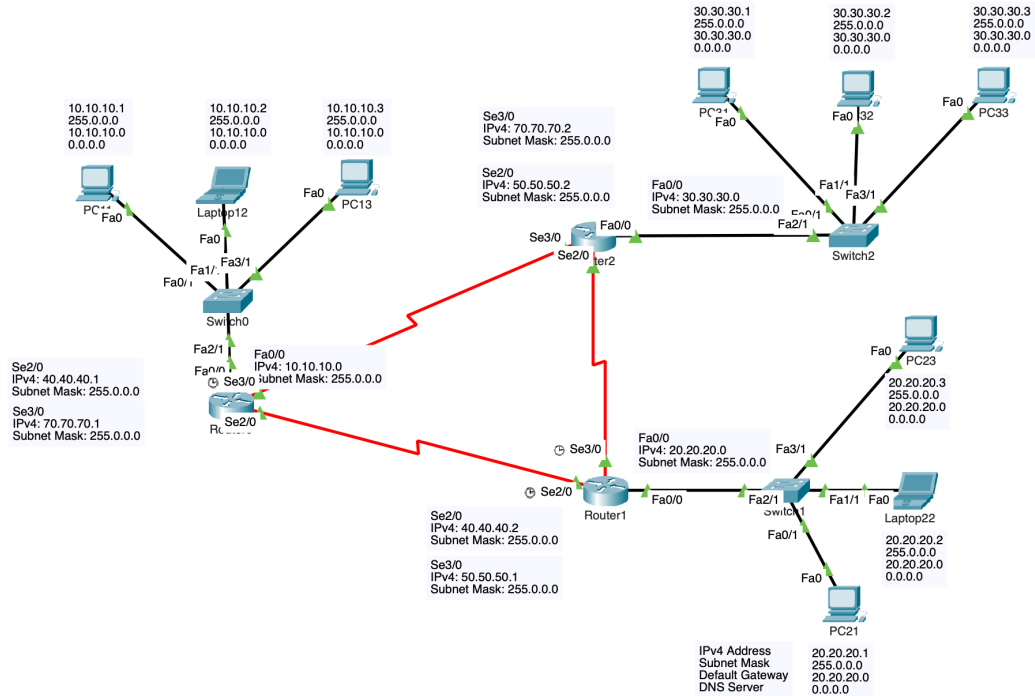


Figure 1: Static Routing Loop Topology

```

Router>enable
Router#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

S    10.0.0.0/8 [1/0] via 70.70.70.1
      [1/0] via 50.50.50.1
S    20.0.0.0/8 [1/0] via 50.50.50.1
C    30.0.0.0/8 is directly connected, FastEthernet0/0
C    50.0.0.0/8 is directly connected, Serial2/0
C    70.0.0.0/8 is directly connected, Serial3/0

Router#

```

Figure 2: Router 2 CLI: Static Routing Table displaying Redundant Paths

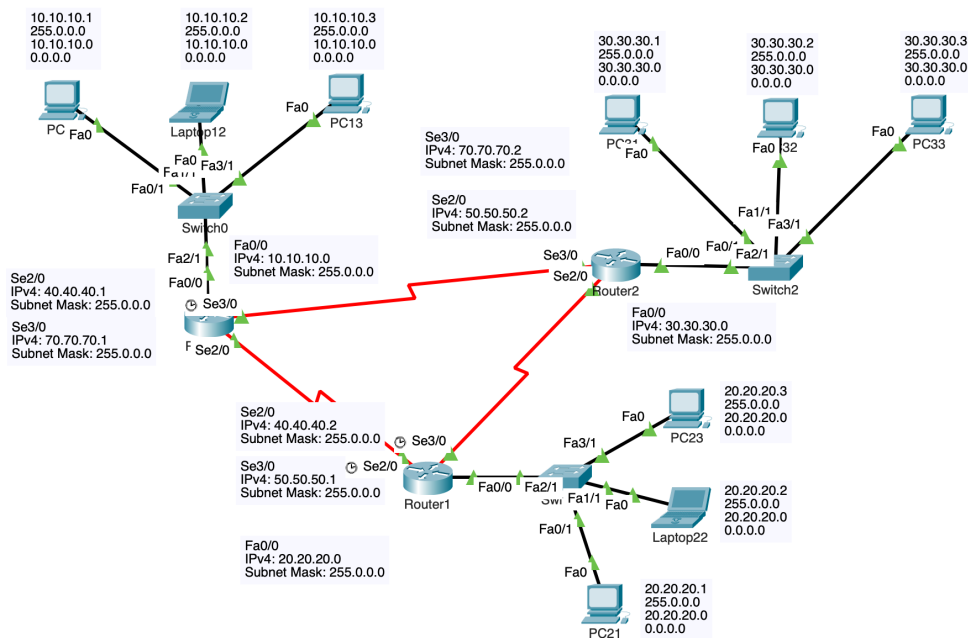


Figure 3: Dynamic RIP Loop Topology (Optimal State)

```

Router>enable
Router#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

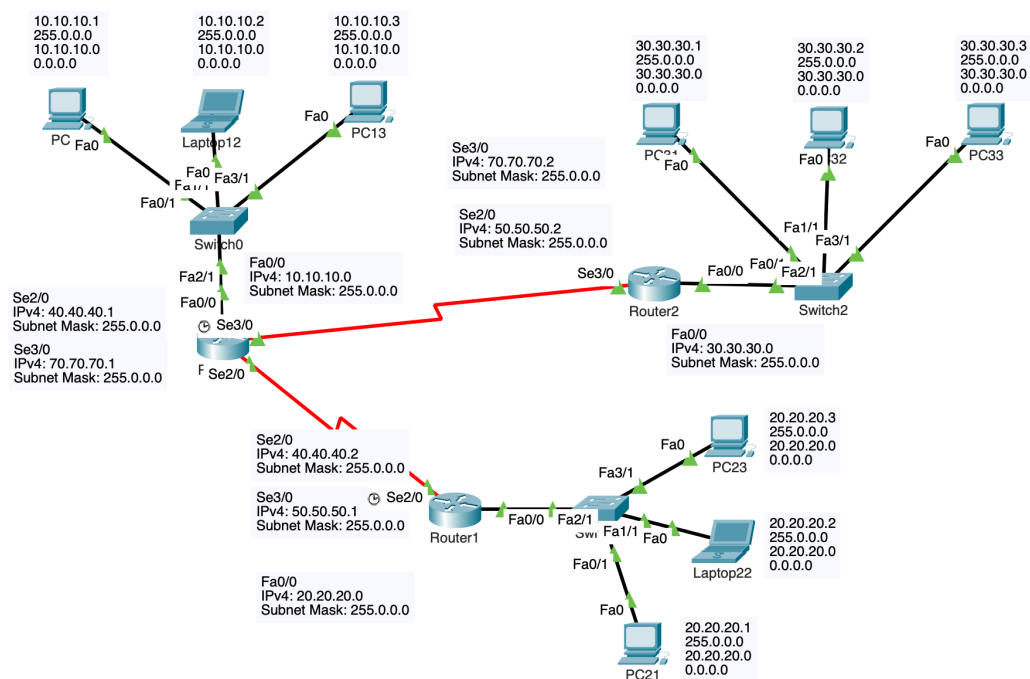
Gateway of last resort is not set

R    10.0.0.0/8 [120/1] via 70.70.70.1, 00:00:09, Serial3/0
R    20.0.0.0/8 [120/1] via 50.50.50.1, 00:00:09, Serial2/0
C    30.0.0.0/8 is directly connected, FastEthernet0/0
R    40.0.0.0/8 [120/1] via 70.70.70.1, 00:00:09, Serial3/0
      [120/1] via 50.50.50.1, 00:00:09, Serial2/0
C    50.0.0.0/8 is directly connected, Serial2/0
C    70.0.0.0/8 is directly connected, Serial3/0

Router#

```

Figure 4: RIP Routing Table (Optimal): Demonstrating Equal-Cost Load Balancing to 40.0.0.0/8



```

Router>enable
Router#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

R    10.0.0.0/8 [120/1] via 40.40.40.1, 00:00:07, Serial2/0
C    20.0.0.0/8 is directly connected, FastEthernet0/0
R    30.0.0.0/8 [120/2] via 40.40.40.1, 00:00:07, Serial2/0
C    40.0.0.0/8 is directly connected, Serial2/0
R    70.0.0.0/8 [120/1] via 40.40.40.1, 00:00:07, Serial2/0

Router#

```

Figure 6: RIP Routing Table (Fault State): Router 1 Dynamically Rerouting via Router 0

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC13	PC31	IC...		0.000	N	0	(...	(delete)
	Successful	PC21	PC32	IC...		0.000	N	1	(...	(delete)
	Successful	PC33	PC11	IC...		0.000	N	2	(...	(delete)
	Successful	Laptop22	PC31	IC...		0.000	N	3	(...	(delete)

Figure 7: Successful ICMP Verification during Link Failure

4 Discussion

The loop topology provided critical insights into network redundancy. During the static routing phase, the routing table for Router 2 (Figure 2) revealed that two explicit pathways were programmed to reach the 10.0.0.0 network: one directly via 70.70.70.1 and an alternate path via 50.50.50.1. This demonstrated that fault tolerance in static environments requires rigorous, manual equal-cost configurations by the administrator.

Conversely, the implementation of the RIP protocol automated this redundancy. As observed in the optimal state (Figure 4), RIP not only found the shortest paths but also successfully performed load balancing. The routing table showed two active paths to the 40.0.0.0 network with an equal metric (hop count of 1), allowing the router to distribute traffic optimally.

The true resilience of the dynamic protocol was tested when the critical link (50.0.0.0 network) between Router 1 and Router 2 was intentionally severed (Figure 5). The protocol's self-healing capabilities were immediately evident in Router 1's updated routing table (Figure 6). Instead of dropping packets destined for the 30.0.0.0 network, RIP dynamically recalculated the path, routing the traffic through Router 0 (via 40.40.40.1) and reflecting an increased hop count of 2 ([120/2]). The subsequent simulation event list (Figure 7) confirmed successful message passing between nodes across the network, proving that RIP maintained seamless end-to-end connectivity during an unexpected physical link failure.

5 Conclusion

The implementation of a loop topology was successfully achieved, demonstrating the importance of physical redundancy in network design. The simulations effectively contrasted static and dynamic routing approaches under failure conditions. While static routing can be forced into redundancy through manual equal-cost configurations, RIP proved highly superior in maintaining seamless end-to-end connectivity during unexpected link failures through automatic path recalculation.

References

- [1] A. S. Tanenbaum, N. Feamster, and D. J. Wetherall, *Computer Networks*, 6th ed. Pearson, 2021.
- [2] W. Stallings, *Data and Computer Communications*, 10th ed. Pearson, 2013.
- [3] *Cisco Packet Tracer 9.0 User Guide*, Cisco Networking Academy, 2026.

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



**Department of
Electrical & Computer Engineering**

Lab Report 5

Study of Static Routing Configuration Using VLSM Technique and Simulation

Course Code: ECE 4212

Course Title: Computer Network Sessional

Submitted to:

Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:

Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Odd

Submission Date: April 19, 2026

Contents

1	Theory and Introduction	1
2	Methodology	1
3	Results	2
3.1	Configuration Notes	2
3.2	Static Route Summary	2
4	Discussion	4
5	Conclusion	5
	References	5

Study of Static Routing Configuration Using VLSM Technique and Simulation

1 Theory and Introduction

Variable Length Subnet Mask (VLSM) is a method used in IP network design to create subnets with varying subnet masks. By allowing different subnet sizes, VLSM enables network administrators to allocate IP addresses more efficiently, using smaller subnet masks for subnets with fewer hosts and larger masks for those with more hosts.

Unlike traditional subnetting (Classful routing), where a fixed subnet mask is applied to all subnets across a network, VLSM provides greater flexibility, drastically reducing IP address waste [1]. It is commonly used in modern networks to optimize IP address usage. Conceptually, VLSM can be described as the process of further subnetting an already existing subnet. In this experiment, a single Class C network block (192.168.1.0/24) was partitioned using VLSM to accommodate four separate Local Area Networks (LANs) and three Wide Area Network (WAN) links using static routing.

2 Methodology

The experiment was conducted using Cisco Packet Tracer (Version 9.0). The static routing network was established through the following systematic steps:

- **VLSM Calculation:** A VLSM allocation table was prepared based on the specific host requirements of the network topology to carve multiple subnets from the 192.168.1.0/24 block.
- **Topology Construction:** The network was designed utilizing three routers. Four distinct LANs were created: Router 0 hosted a 30-host network, Router 1 hosted a 14-host network, and Router 2 hosted two smaller networks for varying host capacities.
- **Interface Configuration:** High-speed serial links were configured between the routers. Point-to-point connections utilized /30 subnet masks to maximize efficiency.
- **Addressing:** Each PC, switch, and router interface was assigned its specific IPv4 address, subnet mask, and default gateway according to the simulated VLSM table.
- **Static Routing:** The global configuration mode on each router was accessed to manually input static routes. The destination network address, VLSM subnet mask, and Next-Hop IP address were explicitly defined.
- **Verification:** The `show ip route` command was utilized to verify the routing tables. End-to-end connectivity was tested by sending ICMP messages between hosts on different subnets.

3 Results

The network was successfully simulated. Table 1 reflects the VLSM subnets as configured in the simulation environment, and Table 2 outlines the device interfaces.

Table 1: Simulated VLSM Subnet Allocation Table

Segment	Network Address	CIDR	Subnet Mask	Usable Host Range	Broadcast Address
LAN 0 (Router 0)	192.168.1.0	/27	255.255.255.224	192.168.1.1 – 192.168.1.30	192.168.1.31
LAN 1 (Router 1)	192.168.1.32	/28	255.255.255.240	192.168.1.33 – 192.168.1.46	192.168.1.47
LAN 2 (Router 2 - A)	192.168.1.48	/29	255.255.255.248	192.168.1.49 – 192.168.1.54	192.168.1.55
LAN 3 (Router 2 - B)	192.168.1.56	/29	255.255.255.248	192.168.1.57 – 192.168.1.62	192.168.1.63
WAN 1 (Router 0 - 1)	192.168.1.56	/30	255.255.255.252	192.168.1.57 – 192.168.1.58	192.168.1.59
WAN 2 (Router 1 - 2)	192.168.1.60	/30	255.255.255.252	192.168.1.61 – 192.168.1.62	192.168.1.63
WAN 3 (Router 2 - 0)	192.168.1.64	/30	255.255.255.252	192.168.1.65 – 192.168.1.66	192.168.1.67

Table 2: Device IPv4 Addressing Summary

Device	Interface	IPv4 Address	Subnet Mask	Default Gateway
Router 0	Fa0/0	192.168.1.1	255.255.255.224	N/A
Router 0	Se2/0	192.168.1.57	255.255.255.252	N/A
Router 0	Se3/0	192.168.1.66	255.255.255.252	N/A
Router 1	Fa0/0	192.168.1.33	255.255.255.240	N/A
Router 1	Se2/0	192.168.1.58	255.255.255.252	N/A
Router 2	Fa0/0	192.168.1.49	255.255.255.248	N/A
Router 2	Fa1/0	192.168.1.57	255.255.255.248	N/A
PC 11 (A)	NIC	192.168.1.2	255.255.255.224	192.168.1.1
PC 21 (B)	NIC	192.168.1.34	255.255.255.240	192.168.1.33
PC 31 (C)	NIC	192.168.1.54	255.255.255.248	192.168.1.49
PC 41 (D)	NIC	192.168.1.58	255.255.255.248	192.168.1.57

3.1 Configuration Notes

The addressing plan was applied from the largest subnet to the smallest subnet to minimize address waste and avoid fragmentation. Each router was configured with an IP address on its LAN-facing interface and appropriate next-hop routes for remote LANs. The serial links were assigned /30 masks because each point-to-point connection requires only two usable host addresses.

3.2 Static Route Summary

To enable inter-network communication, static routes were manually added on each router so that traffic could reach all remote LAN segments through the correct intermediate router. This configuration ensured that packets followed the intended path across the three-router topology and allowed successful ICMP verification between hosts in different subnets.

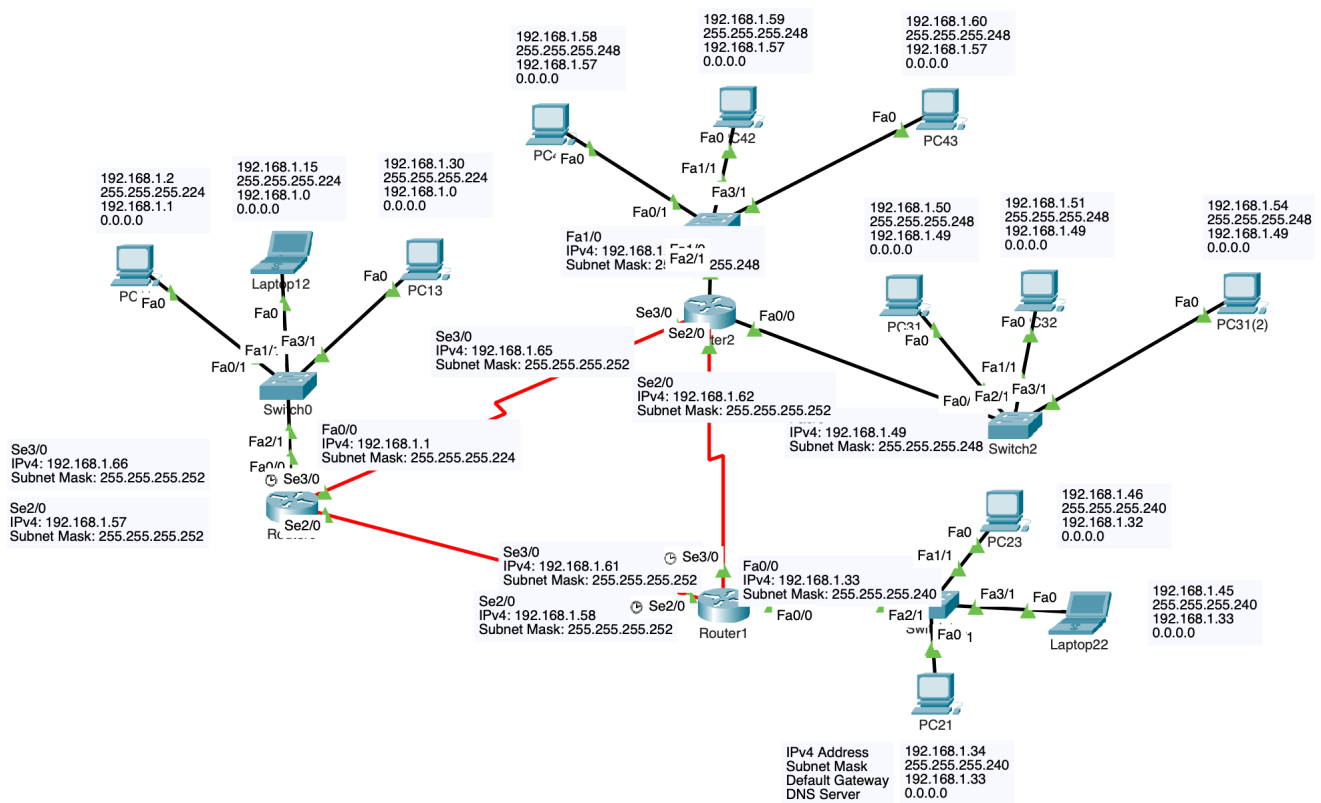


Figure 1: Multi-Router Topology with VLSM Implementation

```
Router#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

    192.168.1.0/24 is variably subnetted, 6 subnets, 4 masks
C       192.168.1.0/27 is directly connected, FastEthernet0/0
S       192.168.1.32/28 [1/0] via 192.168.1.58
S       192.168.1.48/29 [1/0] via 192.168.1.65
S       192.168.1.56/29 [1/0] via 192.168.1.65
C       192.168.1.56/30 is directly connected, Serial2/0
C       192.168.1.64/30 is directly connected, Serial3/0

Router#0
```

Figure 2: Router 0 CLI: Routing Table Output displaying "Variably Subnetted" feature

```

Router#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
        D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
        N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
        E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
        i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
        * - candidate default, U - per-user static route, o - ODR
        P - periodic downloaded static route

Gateway of last resort is not set

192.168.1.0/24 is variably subnetted, 6 subnets, 4 masks
S    192.168.1.0/27 [1/0] via 192.168.1.57
C    192.168.1.32/28 is directly connected, FastEthernet0/0
S    192.168.1.48/29 [1/0] via 192.168.1.62
S    192.168.1.56/29 [1/0] via 192.168.1.62
C    192.168.1.56/30 is directly connected, Serial2/0
C    192.168.1.60/30 is directly connected, Serial3/0
Router#

```

Figure 3: Router 1 Routing Table

```

Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
        D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
        N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
        E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
        i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
        * - candidate default, U - per-user static route, o - ODR
        P - periodic downloaded static route

Gateway of last resort is not set

192.168.1.0/24 is variably subnetted, 6 subnets, 4 masks
S    192.168.1.0/27 [1/0] via 192.168.1.66
S    192.168.1.48/28 [1/0] via 192.168.1.61
C    192.168.1.48/29 is directly connected, FastEthernet0/0
C    192.168.1.56/29 is directly connected, FastEthernet1/0
C    192.168.1.60/30 is directly connected, Serial2/0
C    192.168.1.64/30 is directly connected, Serial3/0
Router#2

```

Figure 4: Router 2 Routing Table

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	Laptop12	PC31	IC...		0.000	N	0	(...	(delete)
	Successful	Laptop12	PC13	IC...		0.000	N	1	(...	(delete)
	Successful	Laptop12	PC21	IC...		0.000	N	2	(...	(delete)
	Successful	PC13	PC42	IC...		0.000	N	3	(...	(delete)
	Successful	PC13	PC43	IC...		0.000	N	4	(...	(delete)

Figure 5: Verification of End-to-End Connectivity between distinct VLSM Subnets

4 Discussion

The implementation of VLSM drastically improved the IP address efficiency of the network. As displayed in the `show ip route` output for Router 0 (Figure 2), Cisco IOS explicitly recognized the single Class C block as being "variably subnetted" into 6 subnets using 4 different masks. Utilizing /30 masks on the serial links was highly effective, conserving address space by providing exactly two IP addresses per point-to-point connection.

During the configuration analysis, two notable routing behaviors were observed that highlight the complexities of static VLSM environments:

1. **Subnet Overlap & Longest Prefix Match:** Table 1 indicates an overlap between LAN 3 (192.168.1.56/29) and WAN 1 (192.168.1.56/30). Because the /30 mask is longer than the /29 mask, Router 0 forwarded traffic destined for PC 41 via the WAN link to Router 1, which then successfully relayed it to Router 2. This demonstrated the "Longest Prefix Match" rule in routing architecture, allowing the ICMP packets to successfully reach their destination (Figure 5) despite the overlap.
2. **Configuration Discrepancy:** The routing table for Router 2 (Figure 4) revealed a static route entry of 192.168.1.48/28 via 192.168.1.61. This entry was likely intended to route traffic to the 192.168.1.32/28 network. Consequently, ICMP requests initiated from Router 2 to Router 1 would fail because the static route was pointing to an incorrect block.

These observations underscore that while VLSM provides unparalleled addressing efficiency, it requires meticulous planning and precise static route implementation to prevent logical overlap and black-hole routing.

5 Conclusion

The study and simulation of static routing utilizing the Variable Length Subnet Mask (VLSM) technique were successfully executed. The experiment confirmed that routers can seamlessly forward traffic between networks of varying sizes—such as /27, /28, and /29 subnets—within a single parent IP block. The analysis of the routing tables proved that VLSM is a fundamental tool for network administrators to conserve IPv4 address space, though it demands careful schematic planning to avoid overlapping ranges in static environments.

References

- [1] A. S. Tanenbaum, N. Feamster, and D. J. Wetherall, *Computer Networks*, 6th ed. Pearson, 2021.

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



**Department of
Electrical & Computer Engineering**

Lab Report 6

Design and Implementation of Error Detection and Correction Mechanisms
using CRC and Hamming Codes

Course Code: ECE 4212

Course Title: Computer Network Sessional

Submitted to:

Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:

Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Even

Submission Date: April 19, 2026

Contents

1	Theory and Introduction	1
2	Methodology and Implementation	1
2.1	CRC Implementation Strategy	1
2.2	Hamming Code Implementation Strategy	2
2.3	CRC Simulation Code	2
2.4	Hamming Code Simulation Code	3
3	Results	4
3.1	CRC Simulation Output	4
3.2	Hamming Code Simulation Output	5
4	Discussion	6
5	Conclusion	6
	References	6

Design and Implementation of Error Detection and Correction Mechanisms using CRC and Hamming Codes

1 Theory and Introduction

In digital communication systems, data transmitted over noisy channels or stored in unreliable media is susceptible to bit errors. Error detection and correction codes are essential mechanisms employed to ensure data integrity [?]. This experiment focuses on two fundamental coding techniques: Cyclic Redundancy Check (CRC) for error detection and Hamming Code for single-bit error correction.

Cyclic Redundancy Check (CRC) is a widely used error-detecting code that appends a short check value (redundancy bits) to a data block. It treats binary data as a polynomial and uses modulo-2 arithmetic (XOR operations) for division [1]. A generator polynomial (key) is agreed upon by both sender and receiver. It is highly effective at detecting burst errors and is commonly used in network protocols like Ethernet.

Hamming Code, developed by Richard Hamming, is an error-correcting code capable of detecting and correcting single-bit errors [2]. This is achieved by inserting multiple parity bits at positions that are powers of two (1, 2, 4, 8, etc.). The number of redundant bits r required for m data bits must satisfy the condition: $2^r \geq m + r + 1$. By recalculating these parity bits at the receiver, the exact position of an erroneous bit can be isolated and rectified [3].

2 Methodology and Implementation

The experiment was conducted by implementing software simulations of the CRC and Hamming Code algorithms in Python. The following processes were executed:

2.1 CRC Implementation Strategy

- **Encoding (Sender):** The original data was appended with $k - 1$ zeros, where k is the length of the generator polynomial. Modulo-2 division was performed on the augmented data using the generator polynomial. The resulting remainder was appended to the original data to form the transmitted codeword.
- **Decoding (Receiver):** Modulo-2 division was performed on the received codeword using the same polynomial. If the final remainder evaluated to zero, the data was accepted as error-free; otherwise, an error was flagged.

2.2 Hamming Code Implementation Strategy

- **Encoding:** The number of required redundant bits was calculated. The data bits were placed in non-power-of-two positions. Each parity bit was calculated to ensure even parity across its designated checking positions.
- **Correction:** At the receiver, all parity bits were recalculated. The XOR of the differing parity positions yielded the error location (syndrome), which was subsequently flipped to restore the original data.

2.3 CRC Simulation Code

The Python code utilized to simulate the detailed, step-by-step operations if CRC is provided below:

```
1 def findXor(a, b):
2     """Performs bitwise XOR between two binary
3     ↪ strings (a and b)."""
4     result = []
5     for i in range(1, len(b)): # Skip first
6     ↪ bit (CRC standard)
7         if a[i] == b[i]:
8             result.append("0")
9         else:
10            result.append("1")
11    return "".join(result)
12
13 def mod2div(dividend, divisor,
14 ↪ show_steps=False):
15     """Performs Modulo-2 division (CRC division
16     ↪ algorithm) with step tracking."""
17     pick = len(divisor)
18     tmp = dividend[0:pick]
19
20     if show_steps:
21         print(f"    [DIV] Initial Dividend:
22         ↪ {dividend}")
23         print(f"    [DIV] Divisor:
24         ↪ {divisor}")
25
26     while pick < len(dividend):
27         if tmp[0] == "1":
28             if show_steps:
29                 print(f"    [DIV] {tmp} XOR
30                 ↪ {divisor} -> ", end="")
31             tmp = findXor(divisor, tmp) +
32             ↪ dividend[pick]
33             if show_steps:
34                 print(f"{tmp} (pulled down
35                 ↪ {dividend[pick]})")
36         else:
37             if show_steps:
38                 print(f"    [DIV] {tmp} XOR
39                 ↪ {'0'*pick} -> ", end="")
40             tmp = findXor("0" * pick, tmp) +
41             ↪ dividend[pick]
42             if show_steps:
43                 print(f"{tmp} (pulled down
44                 ↪ {dividend[pick]})")
45         pick += 1
46     # Final step
47     if tmp[0] == "1":
48         if show_steps:
49             print(f"    [DIV] Final Step:
50             ↪ {tmp} XOR {divisor} -> ",
51             ↪ end="")
52         tmp = findXor(divisor, tmp)
53     else:
54         if show_steps:
55             print(f"    [DIV] Final Step:
56             ↪ {tmp} XOR {'0'*pick} -> ",
57             ↪ end="")
58         tmp = findXor("0" * pick, tmp)
59
60     if show_steps:
61         print(f"{tmp}\n")
62     return tmp
63
64 def encodeData(data, key):
65     """Appends CRC remainder to the original
66     ↪ data."""
67     l_key = len(key)
68     appended_data = data + "0" * (l_key - 1)
69     print(f"--- SENDER SIDE ---")
70     print(f"Original Data: {data}")
71     print(f"Key (Polynomial): {key}")
72     print(f"Appended Data (with {l_key - 1}
73     ↪ zeros): {appended_data}\n")
74
75     print("Calculating Remainder:")
76     remainder = mod2div(appended_data, key,
77     ↪ show_steps=True)
78
79     codeword = data + remainder
80     print(f"Calculated Remainder: {remainder}")
81     print(f"Transmitted Codeword:
82     ↪ {codeword}\n")
83     return codeword
84
85 def receiver(data, key):
86     """Checks if received data has errors
87     ↪ (remainder == 0)."""
```



```

70     print(f"--- RECEIVER SIDE ---")
71     print(f"Received Codeword: {data}\n")
72
73     print("Performing Modulo-2 Division on
74     ↪ Received Data:")
75     remainder = mod2div(data, key,
76     ↪ show_steps=True)
77
78     print(f"Final Remainder: {remainder}")
79     if "1" in remainder:
80         print(
81             "[RESULT] ERROR DETECTED: The
82             ↪ remainder is non-zero. The data
83             ↪ was corrupted during
84             ↪ transmission.\n"
85         )
86     else:
87         print("[RESULT] SUCCESS: The remainder
88         ↪ is zero. No errors detected.\n")
89

```

```

84
85 if __name__ == "__main__":
86     data = "100100"
87     key = "1101"
88     # 1. Sender Encodes
89     codeword = encodeData(data, key)
90     # 2. Receiver Receives Valid Data
91     print(">>> SCENARIO 1: Transmission without
92     ↪ errors")
93     receiver(codeword, key)
94     # 3. Receiver Receives Corrupted Data
95     print(">>> SCENARIO 2: Transmission WITH
96     ↪ errors (Fault Simulation)")
97     # Flip the 3rd bit from '0' to '1'
98     error_codeword = codeword[:2] + "1" +
99     ↪ codeword[3:]
100    print(f"(Simulating noise... flipping a
101    ↪ bit: {codeword} -> {error_codeword}")
102    receiver(error_codeword, key)

```

2.4 Hamming Code Simulation Code

The Python code utilized to simulate the detailed, step-by-step operations if Hamming Code is provided below:

```

1  def calcRedundantBits(m):
2      """Calculate the number of redundant bits
3      ↪ needed."""
4      for i in range(m):
5          if 2**i >= m + i + 1:
6              return i
7
8  def posRedundantBits(data, r):
9      """Place redundant bits at positions that
10     ↪ are powers of two."""
11     j = 0
12     k = 1
13     m = len(data)
14     res = ""
15     for i in range(1, m + r + 1):
16         if i == 2**j:
17             res = res + "0"
18             j += 1
19         else:
20             res = res + data[-1 * k]
21             k += 1
22     return res[::-1]
23
24 def calcParityBits(arr, r, show_steps=False):
25     """Calculate and set parity bits based on
26     ↪ bit positions."""
27     n = len(arr)
28     if show_steps:
29         print("Calculating Parity Bits (Even
30         ↪ Parity):")
31

```

```

30     for i in range(r):
31         val = 0
32         checked_positions = []
33         for j in range(1, n + 1):
34             if j & (2**i) == (2**i):
35                 val = val ^ int(arr[-1 * j])
36                 checked_positions.append(f"pos
37                 ↪ {j} (val {arr[-1 * j]})")
38             # Insert the calculated parity bit back
39             ↪ into the array
40             arr = arr[: n - (2**i)] + str(val) +
41             ↪ arr[n - (2**i) + 1 :]
42
43         if show_steps:
44             print(f" P{2**i} checks: {'',
45             ↪ '.join(checked_positions)}")
46             print(f" => P{2**i} is set to
47             ↪ {val}")
48
49     return arr
50
51 def detectAndCorrectError(arr, r):
52     """Detect error position using parity bits
53     ↪ and correct it."""
54     n = len(arr)
55     res = 0
56     print("\n--- RECEIVER SIDE: Error Checking
57     ↪ ---")
58     print("Recalculating parity bits over
59     ↪ received data:")
60
61     for i in range(r):

```

```

55     val = 0
56     for j in range(1, n + 1):
57         if j & (2**i) == (2**i):
58             val = val ^ int(arr[-1 * j])
59     res = res + val * (10**i)
60     print(f" P{2**i} parity check
61           ↳ evaluates to: {val}")
62     # Convert binary syndrome to decimal
63     error_pos = int(str(res), 2)
64
65     if error_pos == 0:
66         print("\n[RESULT] Syndrome is 0. No
67               ↳ error in the received message.")
68         return arr
69     else:
70         print(
71             f"\n[RESULT] Syndrome is
72             ↳ {str(res).zfill(r)}. Error
73             ↳ detected at position
74             ↳ {error_pos} (from the right).")
75         )
76         # Correct the error
77         arr_list = list(arr)
78         error_index = n - error_pos
79         print(
80             f"-> Fixing error... Flipping bit
81             ↳ at index {error_index} (which
82             ↳ is currently
83             ↳ '{arr_list[error_index]}')")
84         )
85         arr_list[error_index] = "0" if
86         ↳ arr_list[error_index] == "1" else
87         ↳ "1"
88         corrected_arr = "".join(arr_list)
89
90     print(f"-> Corrected Data Codeword:
91           ↳ {corrected_arr}")
92     return corrected_arr
93
94 if __name__ == "__main__":
95     data = "1011001"
96     m = len(data)
97     r = calcRedundantBits(m)
98
99     print("--- SENDER SIDE ---")
100    print(f"Original Data: {data}")
101    print(f>Data Bits (m): {m}")
102    print(f"Redundant Parity Bits required (r):
103          ↳ {r}")
104    # Place '0' in parity positions (1, 2, 4,
105    ↳ 8)
106    arr = posRedundantBits(data, r)
107    print(f>Data with empty parity bits:
108          ↳ {arr}")
109    # Calculate actual parity
110    arr = calcParityBits(arr, r,
111        ↳ show_steps=True)
112    print(f"\nTransmitted Codeword: {arr}")
113    # Simulate Error
114    print("\n>>> Simulating transmission
115          ↳ noise...")
116    # Let's flip the bit at position 7 from the
117    ↳ right (index 4 from left)
118    error_arr = arr[:4] + ("0" if arr[4] == "1"
119        ↳ else "1") + arr[5:]
120    print(f"Original sent: {arr}")
121    print(f"Received      : {error_arr}")
122
123    # Detect and correct
124    corrected_data =
125    ↳ detectAndCorrectError(error_arr, r)

```

3 Results

The implemented Python algorithms were executed to simulate data transmission, error injection, and detection/correction. The verbose console outputs are documented below.

3.1 CRC Simulation Output

```

1  --- SENDER SIDE ---
2  Original Data: 100100
3  Key (Polynomial): 1101
4  Appended Data (with 3 zeros): 100100000
5
6  Calculating Remainder:
7      [DIV] Initial Dividend: 100100000
8      [DIV] Divisor:          1101
9      [DIV] 1001 XOR 1101 -> 1000 (pulled down 0)
10     [DIV] 1000 XOR 1101 -> 1010 (pulled down 0)
11     [DIV] 1010 XOR 1101 -> 1110 (pulled down 0)
12     [DIV] 1110 XOR 1101 -> 0110 (pulled down 0)
13     [DIV] 0110 XOR 00000000 -> 1100 (pulled down 0)
14     [DIV] Final Step: 1100 XOR 1101 -> 001

```

```

15
16 Calculated Remainder: 001
17 Transmitted Codeword: 100100001
18
19 >>> SCENARIO 1: Transmission without errors
20 --- RECEIVER SIDE ---
21 Received Codeword: 100100001
22
23 Performing Modulo-2 Division on Received Data:
24 [DIV] Initial Dividend: 100100001
25 [DIV] Divisor: 1101
26 [DIV] 1001 XOR 1101 -> 1000 (pulled down 0)
27 [DIV] 1000 XOR 1101 -> 1010 (pulled down 0)
28 [DIV] 1010 XOR 1101 -> 1110 (pulled down 0)
29 [DIV] 1110 XOR 1101 -> 0110 (pulled down 0)
30 [DIV] 0110 XOR 00000000 -> 1101 (pulled down 1)
31 [DIV] Final Step: 1101 XOR 1101 -> 000
32
33 Final Remainder: 000
34 [RESULT] SUCCESS: The remainder is zero. No errors detected.
35
36 >>> SCENARIO 2: Transmission WITH errors (Fault Simulation)
37 (Simulating noise... flipping a bit: 100100001 -> 101100001)
38 --- RECEIVER SIDE ---
39 Received Codeword: 101100001
40
41 Performing Modulo-2 Division on Received Data:
42 [DIV] Initial Dividend: 101100001
43 [DIV] Divisor: 1101
44 [DIV] 1011 XOR 1101 -> 1100 (pulled down 0)
45 [DIV] 1100 XOR 1101 -> 0010 (pulled down 0)
46 [DIV] 0010 XOR 000000 -> 0100 (pulled down 0)
47 [DIV] 0100 XOR 00000000 -> 1000 (pulled down 0)
48 [DIV] 1000 XOR 1101 -> 1011 (pulled down 1)
49 [DIV] Final Step: 1011 XOR 1101 -> 110
50
51 Final Remainder: 110
52 [RESULT] ERROR DETECTED: The remainder is non-zero. The data was corrupted during transmission.

```

3.2 Hamming Code Simulation Output

```

1 --- SENDER SIDE ---
2 Original Data: 1011001
3 Data Bits (m): 7
4 Redundant Parity Bits required (r): 4
5 Data with empty parity bits: 10101000100
6 Calculating Parity Bits (Even Parity):
7 P1 checks: pos 1 (val 0), pos 3 (val 1), pos 5 (val 0), pos 7 (val 1), pos 9 (val 1), pos 11 (val
  ↩ 1)
8 => P1 is set to 0
9 P2 checks: pos 2 (val 0), pos 3 (val 1), pos 6 (val 0), pos 7 (val 1), pos 10 (val 0), pos 11 (val
  ↩ 1)
10 => P2 is set to 1
11 P4 checks: pos 4 (val 0), pos 5 (val 0), pos 6 (val 0), pos 7 (val 1)
12 => P4 is set to 1
13 P8 checks: pos 8 (val 0), pos 9 (val 1), pos 10 (val 0), pos 11 (val 1)
14 => P8 is set to 0
15
16 Transmitted Codeword: 10101001110
17

```

```

18 >>> Simulating transmission noise...
19 Original sent: 10101001110
20 Received      : 10100001110
21
22 --- RECEIVER SIDE: Error Checking ---
23 Recalculating parity bits over received data:
24     P1 parity check evaluates to: 1
25     P2 parity check evaluates to: 1
26     P4 parity check evaluates to: 1
27     P8 parity check evaluates to: 0
28
29 [RESULT] Syndrome is 0111. Error detected at position 7 (from the right).
30 -> Fixing error... Flipping bit at index 4 (which is currently '0')
31 -> Corrected Data Codeword: 10101001110

```

4 Discussion

The simulations provided a detailed operational view of both error-handling mechanisms. During the CRC analysis, the modulo-2 division was observed step-by-step. It was verified that appending the calculated remainder (001) to the original data resulted in a codeword (100100001) that became perfectly divisible by the polynomial (1101), yielding a final remainder of 000 at the receiver. When an intentional fault was introduced (flipping the 3rd bit), the division logic was disrupted, producing a non-zero remainder (110), thereby successfully alerting the system to the corruption.

In the Hamming Code analysis, the capability to not only detect but correct a fault was demonstrated. Four parity bits (P_1, P_2, P_4, P_8) were utilized to secure the 7-bit data block. Upon intentionally corrupting the 7th bit from the right, the receiver's recalculation of the parities yielded a syndrome of 0111 (decimal 7). This syndrome accurately pinpointed the precise location of the error, allowing the algorithm to automatically flip the bit and reconstruct the original transmitted codeword (10101001110) without requiring data retransmission.

5 Conclusion

The design and implementation of error detection and correction mechanisms were successfully achieved using Python simulations. The CRC method proved highly efficient for detecting errors utilizing modulo-2 arithmetic, making it ideal for network protocols where retransmission is acceptable. Conversely, the Hamming Code implementation successfully localized and rectified single-bit errors through parity synchronization. This confirmed that while Hamming Code introduces a larger logarithmic overhead, its forward error correction (FEC) capabilities are invaluable in scenarios where retransmission is costly or impossible.

References

- [1] GeeksforGeeks, "Cyclic redundancy check and modulo-2 division," 2025, accessed: 2026-04-18. [Online]. Available: <https://www.geeksforgeeks.org/dsa/modulo-2-binary-division/>
- [2] R. W. Hamming, "Error detecting and error correcting codes," *Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, 1950.
- [3] GeeksforGeeks, "Hamming code implementation in python," 2025, accessed: 2026-04-18. [Online]. Available: <https://www.geeksforgeeks.org/python/hamming-code-implementation-in-python/>